CHROMA LAB

AI KNOWLEDGE WORKSPACE

DETAILED PROJECT REPORT

Retrieval-Augmented Generation (RAG) with FastAPI, ChromaDB and Groq

ACADEMIC PROJECT DOCUMENTATION

Prepared for project review and submission

Project at a Glance

Project name	Chroma Lab — AI Knowledge Workspace
Project category	AI-powered document question answering
Architecture	Web frontend + FastAPI backend + RAG pipeline
Core technologies	HTML, CSS, JavaScript, Python, FastAPI, ChromaDB, Sentence Transformers, Groq API
Current verification	Local backend, document listing and sample RAG question tested
Deployment status	Public deployment not yet confirmed; deployment is a future milestone

Executive Summary

Chroma Lab is an AI Knowledge Workspace designed to let users ask natural-language questions about a collection of documents. It combines a browser-based interface with a Python FastAPI backend, ChromaDB vector storage, text embeddings, and a Groq-hosted language model.

The project follows a Retrieval-Augmented Generation (RAG) approach: relevant document chunks are retrieved first and then supplied as context for answer generation. This helps make responses more relevant to the indexed knowledge set and provides source information where available.

During local development, the backend API, document listing, and a sample RAG question were tested successfully. The project is currently described as a locally working application; public deployment, production reliability, and formal performance testing should be treated as future work until separately verified.

1. Introduction

Modern users often keep useful information in PDFs, text files, Markdown notes, guides, and other documents. Finding a specific answer may require opening multiple files, searching manually, and combining information across pages. Chroma Lab explores a more direct interaction model: add knowledge files, ask a question in plain language, and review an answer with source context.

The project is both a web application and a practical demonstration of a RAG pipeline. It connects user experience, backend API design, vector retrieval, and language-model generation in one workflow.

2. Problem Statement

Document collections can become difficult to search as their size grows. Keyword search may miss semantically related passages, while a general-purpose language model may answer without access to a user's private or project-specific documents. A useful knowledge assistant needs a way to locate relevant passages and use those passages as context for an answer.

Chroma Lab addresses this challenge by indexing document chunks in a vector database and retrieving relevant chunks for each question before generating a response.

3. Project Objectives

• Build a clean, responsive interface for document-based question answering.

• Provide document upload and ingestion into a searchable knowledge store.

• Split extracted text into manageable chunks suitable for embedding and retrieval.

• Use semantic search to retrieve context relevant to a natural-language question.

- Generate a response using retrieved context and a Groq-hosted language model.
- Display document inventory and source information to support verification.
- Keep the frontend and backend modular so the application can be tested and deployed independently.

4. Scope and Main Features

Document workspace: the interface groups navigation, backend status, knowledge statistics, upload controls, indexed documents, and chat into a single workspace.

Document ingestion: uploaded or configured knowledge files are processed by the backend's ingestion service. Exact format support depends on the loaders enabled in the project.

Vector indexing: document text is divided into chunks, embedded, and stored in ChromaDB with metadata such as source names.

RAG question answering: a question is sent to the API; relevant chunks are retrieved and supplied to the language model to support a response.

Document inventory: the GET /documents route reports indexed source names and chunk counts.

Operational routes: the API exposes health, upload, reset, and RAG query routes.

5. Technology Stack

Python: backend language and application logic.

FastAPI: HTTP API framework used to expose backend endpoints and interactive API documentation.

HTML: frontend structure and interface content.

CSS: visual theme, layout, spacing, responsive rules, and component styling.

JavaScript: frontend interactions, document listing, upload/chat requests, and API response rendering.

ChromaDB: persistent vector database used to store and search embedded document chunks.

Sentence Transformers: embedding model family used to turn text into vectors for semantic similarity search; the current configuration references all-MiniLM-L6-v2.

Groq API: language-model inference endpoint used to generate answers from the retrieved context.

Note: versions and runtime settings should be confirmed from the final requirements/configuration files before the project is released.

6. System Architecture

The system can be understood as five connected layers. First, the user interacts with the browser-based frontend. Second, JavaScript sends HTTP requests to the FastAPI application. Third, backend services coordinate ingestion or question answering. Fourth, ChromaDB stores and retrieves document chunks using vector similarity. Fifth, the retrieved context is passed to the configured Groq model, and the resulting answer is returned to the frontend.

Two principal workflows operate within this architecture. The ingestion workflow extracts text, divides it into chunks, creates embeddings, and stores the chunks and metadata. The question-answering workflow accepts a question, retrieves relevant chunks, assembles context, invokes the language model, and returns an answer with source information when available.

Keeping these responsibilities separated makes the system easier to troubleshoot: an upload problem can be investigated independently from retrieval, model access, or frontend rendering.

7. Retrieval-Augmented Generation (RAG)

RAG combines information retrieval with text generation. Instead of asking the model to respond only from its pre-trained knowledge, the application first searches a selected knowledge collection for relevant text. The retrieved material is then included in the prompt or context provided to the model.

Step 1 — Embed and index: document chunks are converted into numerical vectors and saved in the vector store alongside their metadata.

Step 2 — Retrieve: the user's question is embedded or otherwise prepared for search, and the vector store returns the most relevant chunks according to the configured retrieval settings.

Step 3 — Augment: the retrieved passages are included as context for the language model.

Step 4 — Generate and return: the model produces a response, and the backend returns it to the frontend with source details where supported.

RAG can improve grounding, but it does not guarantee correctness. Results depend on source quality, chunk boundaries, embedding relevance, retrieval settings, prompts, and model behavior. Users should verify important claims against the original documents.

8. Backend API Overview

GET /health — provides a simple backend health response.

GET /documents — returns the indexed document list, including source names and chunk counts.

POST /upload — accepts document uploads and triggers the configured ingestion workflow.

POST /reset — resets the active collection according to the backend implementation.

POST /rag/query — accepts a question and retrieval options, searches the knowledge base, and returns a generated answer.

The local FastAPI interactive documentation is available at http://127.0.0.1:8000/docs while the development server is running. This is a local development address, not a public deployment URL.

9. Frontend and User Experience

The frontend is organized around a knowledge workspace. A sidebar provides navigation, backend status, knowledge statistics, upload controls, and a list of indexed documents. The central workspace presents an introduction, suggested prompts, conversation history, and the message composer. The design uses a dark navy foundation with cyan accents to create a consistent technical visual identity.

Recent styling work focused on reducing excess spacing and balancing the sidebar, welcome area, suggestion cards, chat history, and composer. The intention is to give the conversation area more usable space without changing the existing JavaScript behavior or backend API.

Responsive behavior should be verified at common desktop, tablet, and mobile widths before release. Browser hard-refresh may be needed after replacing a stylesheet so cached CSS does not obscure changes.

10. Testing and Observed Results

Local backend startup: the FastAPI application was run in the development environment and its interactive API documentation was accessible.

Document inventory: the GET /documents endpoint returned an indexed source named chromadb-notes.txt with one chunk in the recorded test state.

RAG sample question: the question “What is ChromaDB?” returned the response “ChromaDB is a vector database used to store and search embeddings.”

Frontend integration: the user confirmed that the chat returned a correct answer after testing.

These checks demonstrate a working basic local path from question to retrieval and answer generation. They are not a substitute for automated tests, security review, upload edge-case testing, concurrency testing, or load testing.

11. Security, Privacy, and Reliability Considerations

API key protection: the Groq API key must remain in a backend environment variable and must never be placed in browser JavaScript or committed to a public repository.

Environment configuration: use a local .env file for development, ensure it is ignored by Git, and configure deployment secrets through the hosting provider's environment-variable settings.

Upload validation: production deployments should enforce file type and size limits, handle malformed files safely, and return clear error messages.

Data isolation: if multiple users will use the application, design authentication and per-user collection or metadata isolation before storing private documents.

Reset behavior: confirm the reset endpoint only deletes the intended collection and provide a clear confirmation in the UI if destructive operations are exposed.

Reliability: add timeouts, retry handling where appropriate, structured logs, and user-friendly error states for model/API failures.

Source transparency: encourage users to open original documents when validating important information. A source label does not automatically prove that every generated statement is supported.

12. Limitations

The current project has been validated as a local development application; a public production deployment has not been confirmed in this report.

Answer quality may vary depending on document extraction, chunking, embedding model behavior, similarity search, context length, prompts, and language-model output.

Only the file types supported by the configured loaders can be processed successfully.

Local persistence and filesystem paths may behave differently on serverless or ephemeral hosting platforms. The ChromaDB storage strategy should be checked before choosing a deployment target.

Formal measurements such as response latency, retrieval precision, answer faithfulness, and concurrent-user capacity have not been established.

13. Future Enhancements

Version-controlled source code and a complete README with setup, environment variables, architecture, and screenshots.

Automated tests for API routes, upload validation, document listing, reset behavior, and RAG responses.

Public deployment of the frontend and backend with a correct API base URL, CORS configuration, persistent storage, and server-side secrets.

Authentication and per-user knowledge spaces if private multi-user use is required.

Improved source citations that display filenames and, where possible, page or chunk details.

Progress indicators, loading states, retry options, empty states, and clearer error handling.

Evaluation dataset and metrics for retrieval relevance, answer faithfulness, latency, and failure cases.

Additional file types, document deletion, re-indexing, filters, and conversation export where useful.

14. Conclusion

Chroma Lab demonstrates a practical document-question-answering workflow using a web interface, FastAPI, ChromaDB, text embeddings, and a Groq-hosted language model. Its central value is the combination of semantic retrieval and natural-language generation over a selected document collection.

The core local workflow has been tested successfully, including document inventory and a sample grounded answer. The next stage is to harden edge cases, document the setup, push the code to GitHub, and deploy the frontend and backend with secure configuration and suitable persistent storage.

Appendix A — Key Terms

Term	Meaning
RAG	Retrieval-Augmented Generation: retrieves relevant content before generating an answer.
Embedding	A numerical representation of text used to compare semantic similarity.
Vector database	A database designed to store vectors and search for similar vectors.
Chunk	A smaller segment of a document indexed independently for retrieval.
API	A defined interface that lets frontend and backend software exchange requests and responses.
Source metadata	Information such as a filename attached to stored document chunks.